
CS 104.3 -IT Solution Architecture

Lesson 3-Addressing Modes

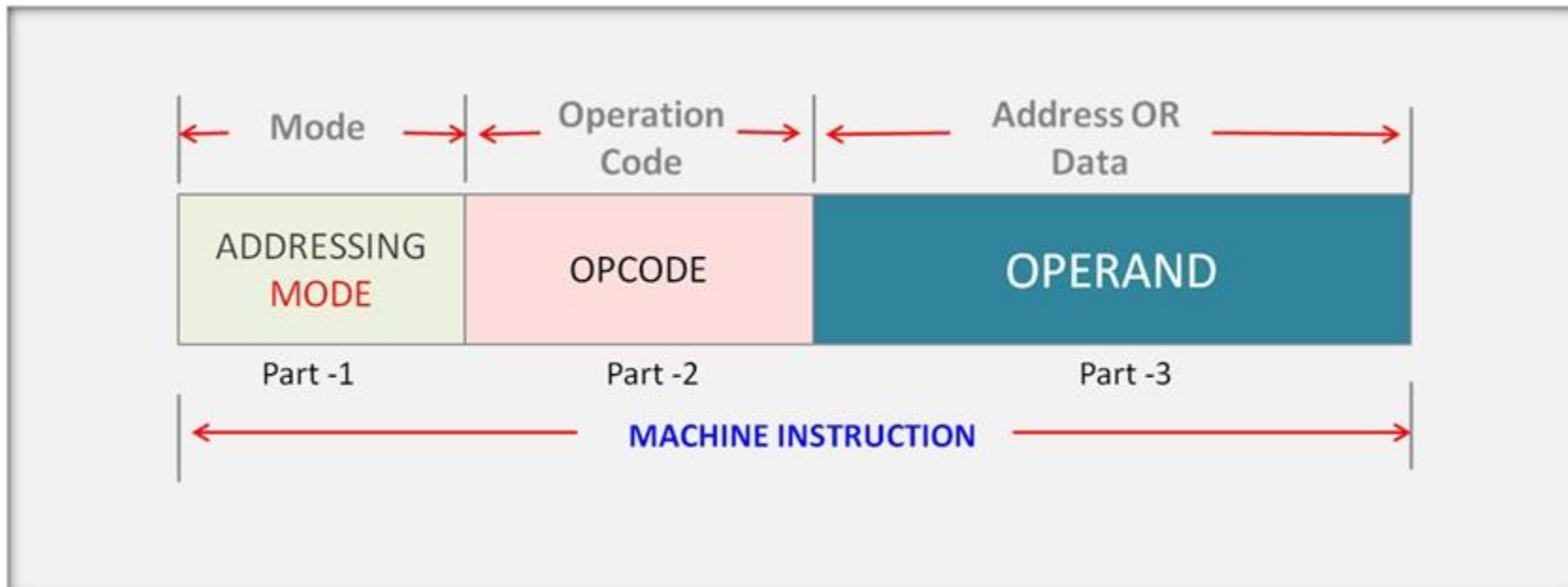
Wake up



Addressing modes

- An addressing mode is a way for a CPU to locate the data (operand) that an instruction needs to work with.
- An **addressing mode** tells the processor **how to find the operands** (data values) used in an instruction, whether the data is inside the instruction itself, in a register, or memory.

Example



Note :- the instruction format may vary depends on the processor architecture

TYPES

1. Implied Addressing mode
 2. Immediate Addressing mode
 3. Register Addressing mode
 4. Direct Addressing mode
 5. Indirect Addressing mode
 6. Indexed Addressing mode
-

Implied

- In Implied (or Implicit) Addressing Mode, the operand is not explicitly mentioned in the instruction
- It is understood or built into the instruction itself.
- Makes instructions shorter and faster to decode
- Example
 - CLR
 - Clear the accumulator

Immediate

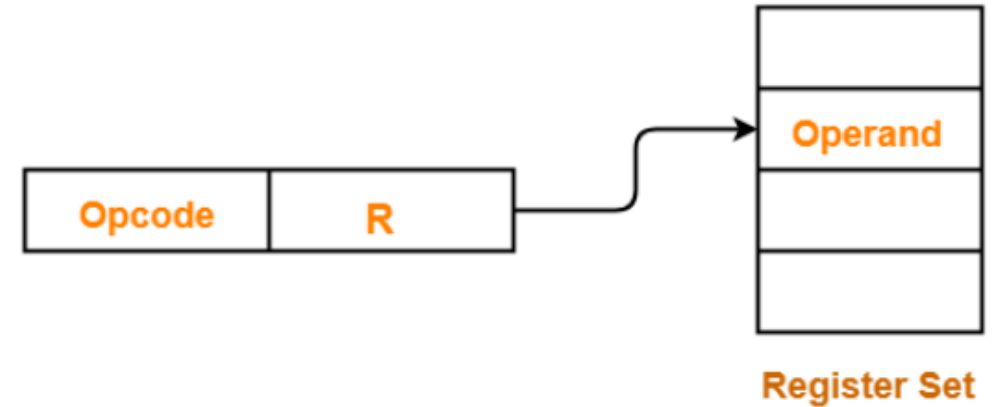
- In this addressing mode, the **operand is specified in the instruction** explicitly.
- Instead of address field, an operand field is present that contains the operand.



- Example
 - **ADD #10**
 - will increment the value stored in the accumulator by 10.

Register

- Register Addressing Mode is a mode in which the **operand is located in a CPU register**.
- Example
 - `ADD R3, R1, R2`



Exercise

- Identify the addressing mode.

Instruction	Addressing Mode
MOV R1, R2	?
MOV R1, #25	?
CLR	?
ADD R2, R3	?
MOV R4, #100	?
INC	?
SUB R1, R5	?
HLT	?

EFFECTIVE ADDRESS(EA)

Effective Address (EA) is the actual memory address where the operand/data is located.

Effective Address = the address calculated/found by the CPU to access the actual data.

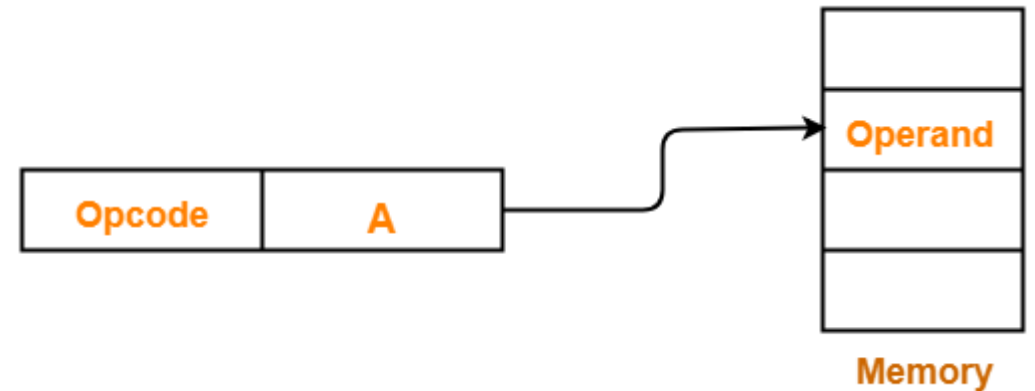
2 min break

- Stand up if you checked your phone in the last 10 minutes.



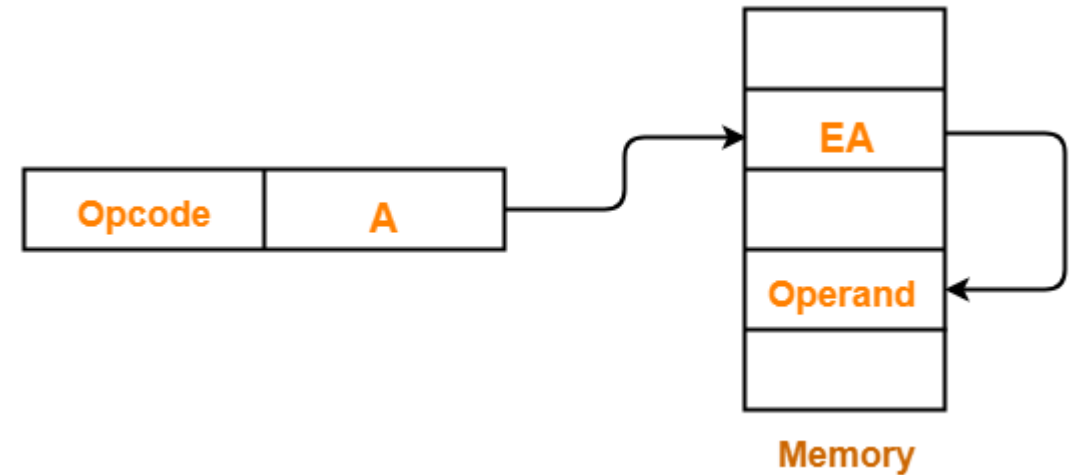
Direct(absolute)

- In this addressing mode, the address field of the instruction contains the effective address of the operand.
- It is also called as **absolute addressing mode**.
- Example
 - **LOAD R1, [100]**
 - Load the content from 100 memory address to the R1



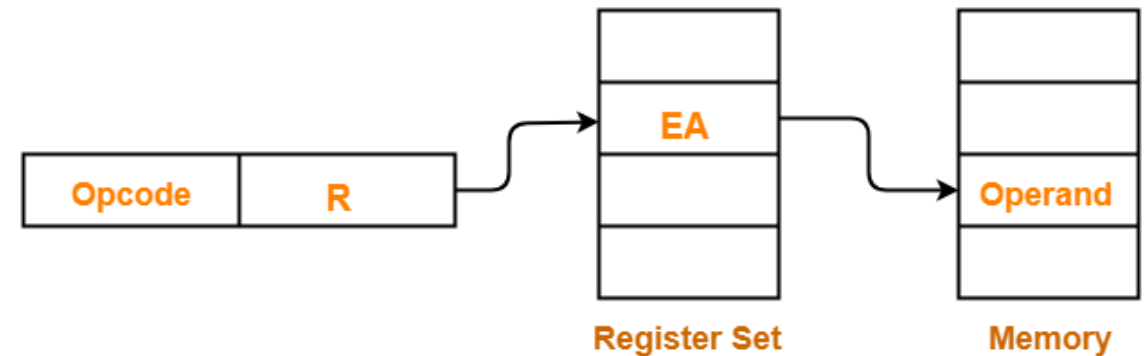
Indirect

- In this addressing mode, the address field of the instruction specifies the address of memory location that contains the **effective address** of the operand.
- Example
 - **LOAD R1, [[100]]**



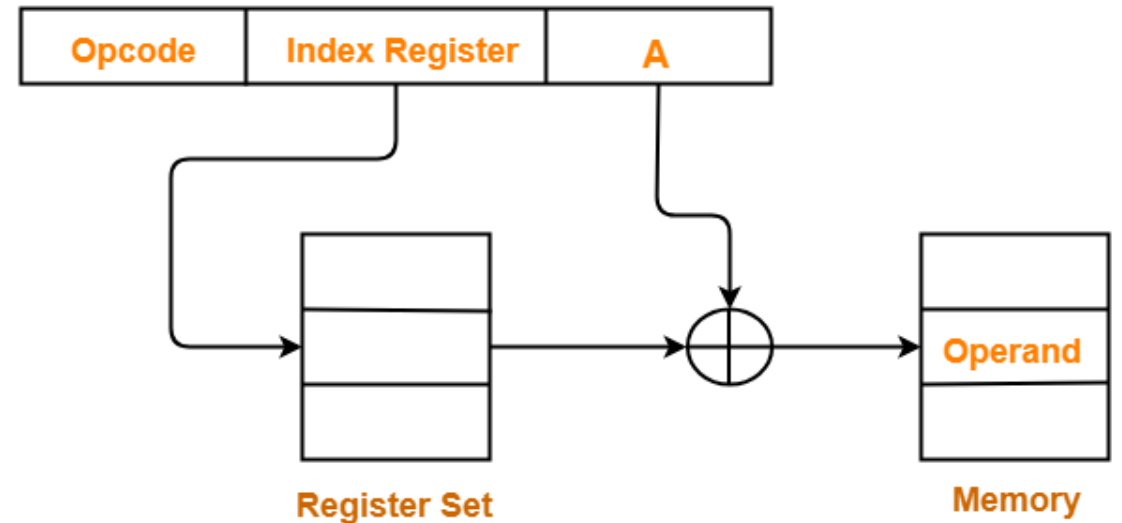
Indirect -> Register Indirect

- The address field of the instruction refers to a CPU register that contains the effective address of the operand.
- Example
 - `LOAD A, [R2]`



Indexed

- Effective address of the operand is obtained by adding the **content of index register** with the **address part** of the instruction.
- Example
 - Assume XR is the indexed register(a register used to modify or calculate a memory address.)
 - `MOV R1, [1000 + XR]`



Exercise

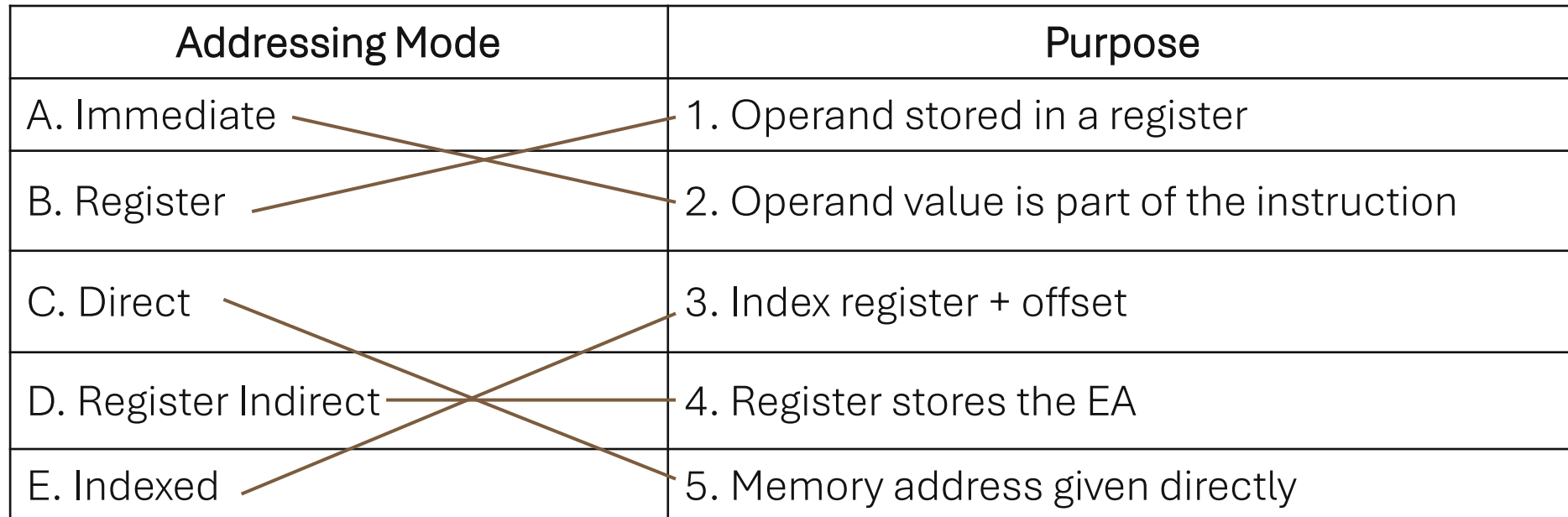
- Match

Addressing Mode	Purpose
A. Immediate	1. Operand stored in a register
B. Register	2. Operand value is part of the instruction
C. Direct	3. Index register + offset
D. Register Indirect	4. Register stores the memory address
E. Indexed	5. Memory address given directly

Exercise

- Match

Addressing Mode	Purpose
A. Immediate	1. Operand stored in a register
B. Register	2. Operand value is part of the instruction
C. Direct	3. Index register + offset
D. Register Indirect	4. Register stores the EA
E. Indexed	5. Memory address given directly



Exercise

- Determine the operand value

Instruction	Operand Value
1. LOAD A, #110	_____
2. LOAD A, R1	_____
3. LOAD A, [100]	_____
4. LOAD A, [R2]	_____
5. LOAD A, [XR+10]	_____

CPU Register	
A	10
R0	20
R1	30
R2	100
XR	110

Memory	
100	100

110	200

120	300

Exercise

- Calculate both effective address and the operand value.

Instruction	Effective Address	Operand Value
1. LOAD A, [100]	_____	_____
2. LOAD A, [110]	_____	_____
3. LOAD A, [XR+10]	_____	_____
4. LOAD A, [[150]]	_____	_____
5. LOAD A, [R2]	_____	_____

CPU Register	
A	10
R0	20
R1	30
R2	100
XR	110

Memory	
100	100

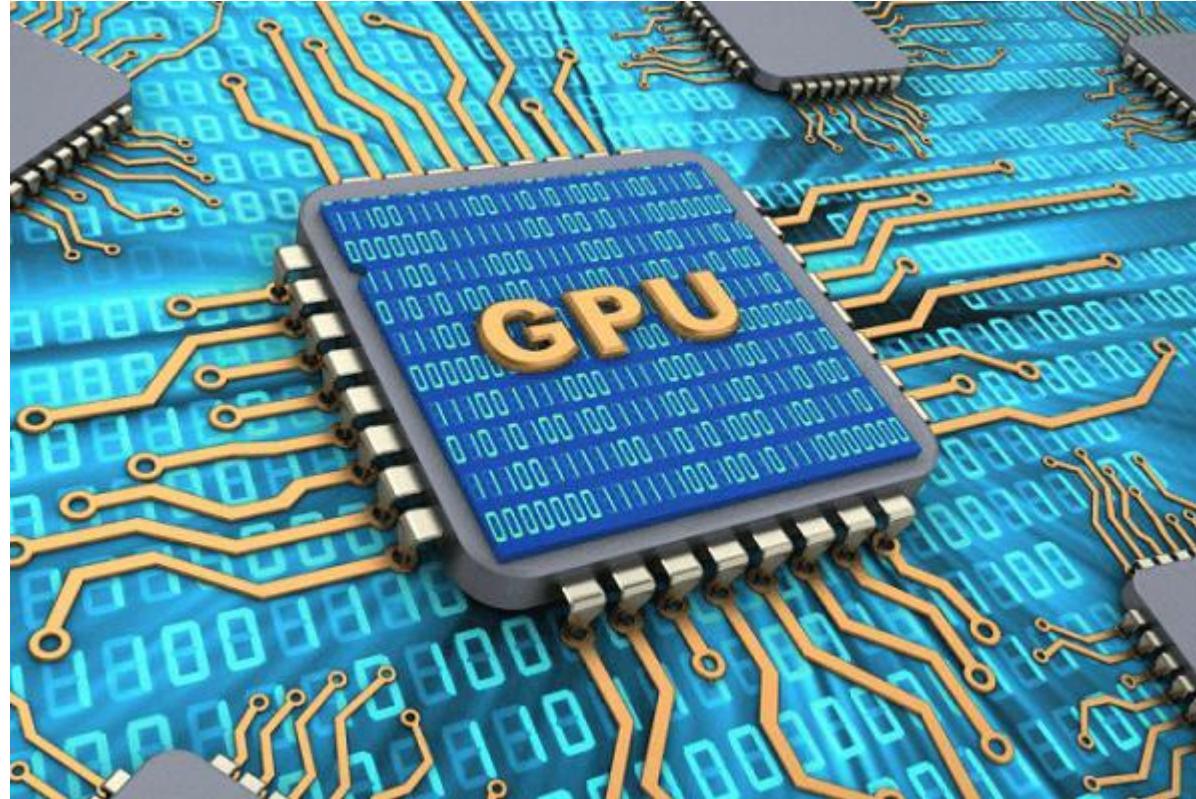
110	200

120	300

150	1000

1000	45

GPU



[CPU vs GPU | Simply Explained](#)

Thank You!
